CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 13

AJAX Event CRUD with the JavaScript Fetch API and a PHP JSON API

PHP 8 · MySQLi · MySQL (MariaDB) · JavaScript Fetch API · JSON

Student Name	Shreyas
Roll Number	25CS023
Student ID	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

1. Problem Definition and Objectives

In a classic PHP application every action submits a form and reloads the whole page. An event list that is edited often feels slow that way and loses the user's position and search. The aim of this practical is to manage events **asynchronously**: the browser sends small requests with the JavaScript **Fetch API** to a PHP endpoint that answers in **JSON**, and the page updates only the affected part – the page is never reloaded.

- List events from MySQL, add, edit/update and delete events with Fetch requests to `api.php` – without reloading the page.
- Return valid JSON for every request, handle GET (read) and POST (change) correctly and answer other methods with an error.
- Validate all input on the server before any database operation; use MySQLi prepared statements; output data safely.
- Handle errors centrally in JSON and show loading, success and error states in the page.
- Intermediate extension: an AJAX search with **debounce** (request sent only after the user stops typing).

Starting point: the Practical-13 folder contained an older version built with PDO on the shared `student_db` database (students and a role-based menu). It was backed up and replaced by the event application described here, which uses MySQLi and its own database `studenthub_prac13`. The five required file names were kept.

2. Tools and Technologies

Tool	Use in this practical
PHP 8.2.12 (XAMPP), MySQLi	JSON API (api.php), database connection (db.php), prepared statements
MariaDB 10.4.32 (XAMPP)	Database studenthub_prac13; schema.sql exported with mysqldump
HTML5, CSS3	index.html (table, form, dialog), style.css
JavaScript (ES2020) – Fetch API	script.js: fetch(), async/await, JSON, AbortController, debounce
Microsoft Edge + DevTools	Running the page; Network panel evidence; headless Edge (Playwright) for the browser test
Python 3	tests/test_app.py – automated API tests (standard library only)

3. Required Source Files

File	Purpose
db.php	MySQLi connection to studenthub_prac13; settings from defaults, config.local.php or P13_* variables; errors logged, never shown
api.php	JSON API: list/get (GET), create/update/delete (POST); validation; prepared statements; central JSON error handling
index.html	Front end: event table, add/edit form, search box, category filter, loading indicator, messages, delete dialog
script.js	Fetch API: apiRequest() wrapper, table rendering with textContent, debounced search, loading/success/error states
schema.sql	Importable schema of studenthub_prac13 with 12 fictional events (mysqldump export)

Helper files: `style.css`, `favicon.svg`, `router.php` (rules for PHP's built-in server), `start-server.bat`, `.htaccess`, `config.example.php`, `.gitignore`.

4. Database Schema

`studenthub_prac13` contains one InnoDB table, `events`. CHECK constraints repeat the most important rules inside MySQL (category list, capacity 1–5000, title at least 3 characters); indexes support sorting by date and searching.

Column	Type	Rules
event_id	INT UNSIGNED AUTO_INCREMENT	Primary key
title	VARCHAR(120)	3–120 characters; index
description	VARCHAR(1000)	10–1000 characters
category	VARCHAR(30)	Cultural, Hackathon, Seminar, Sports, Technical, Workshop; index
event_date / start_time	DATE / TIME	valid date 2000–2100, time HH:MM; composite index
venue	VARCHAR(120)	2–120 characters
capacity	SMALLINT UNSIGNED	1–5000
status	ENUM('open','closed')	default open
created_at / updated_at	DATETIME	updated_at changes on every UPDATE

schema.sql (lines 43–62) – events table (12 fictional seed events: 9 open, 3 closed)

```
43  CREATE TABLE `events` (
44    `event_id` int(10) unsigned NOT NULL AUTO_INCREMENT,
45    `title` varchar(120) NOT NULL,
46    `description` varchar(1000) NOT NULL,
47    `category` varchar(30) NOT NULL,
48    `event_date` date NOT NULL,
49    `start_time` time NOT NULL,
50    `venue` varchar(120) NOT NULL,
51    `capacity` smallint(5) unsigned NOT NULL,
52    `status` enum('open','closed') NOT NULL DEFAULT 'open',
53    `created_at` datetime NOT NULL DEFAULT current_timestamp(),
54    `updated_at` datetime NOT NULL DEFAULT current_timestamp() ON UPDATE current_timestamp(),
55    PRIMARY KEY (`event_id`),
56    KEY `idx_events_date` (`event_date`,`start_time`),
57    KEY `idx_events_category` (`category`),
58    KEY `idx_events_title` (`title`),
59    CONSTRAINT `chk_events_category` CHECK (`category` in
     ('Cultural','Hackathon','Seminar','Sports','Technical','Workshop')),
60    CONSTRAINT `chk_events_capacity` CHECK (`capacity` between 1 and 5000),
61    CONSTRAINT `chk_events_title` CHECK (char_length(`title`) >= 3)
62  ) ENGINE=InnoDB AUTO_INCREMENT=13 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.1 Running the application

- Start MySQL in the XAMPP Control Panel.
- Import: phpMyAdmin → Import → `schema.sql`, or `C:\xampp\mysql\bin\mysql.exe -u root < schema.sql` in the Practical-13 folder.
- Start: `start-server.bat`, or `C:\xampp\php\php.exe -S 127.0.0.1:8613 router.php` in the Practical-13 folder.
- Open **http://127.0.0.1:8613/index.html** in Microsoft Edge; F12 → Network → Fetch/XHR shows every request to `api.php`.

5. AJAX CRUD Implementation

5.1 PHP JSON API (api.php)

`api.php` reads `action` from the query string. `list` and `get` require GET; `create`, `update` and `delete` require POST with a JSON body (`Content-Type: application/json` and the header `X-Requested-With: fetch`).

Every answer leaves through one function, `respond()`, which sets the status code and JSON headers. Expected problems are thrown as `ApiError` (400, 404, 405, 415, 422); anything else is caught by the central exception handler, written to the PHP error log and answered with a general 500 message – no SQL or connection details reach the browser.

api.php (lines 29–52) – single JSON exit point and central error handling

```
29  // The single exit point: sends the status code, JSON headers and the JSON body
30  function respond(int $status, array $body): never
31  {
32      http_response_code($status);
33      header('Content-Type: application/json; charset=utf-8');
34      header('Cache-Control: no-store');
35      header('X-Content-Type-Options: nosniff');
36      echo json_encode($body, JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES | JSON_HEX_TAG |
     JSON_HEX_AMP | JSON_HEX_APOS | JSON_HEX_QUOT);
37      exit;
38  }
39
40  // Centralized error handling: expected problems are ApiError, everything else is logged and answered
     with 500
41  set_exception_handler(function (Throwable $e): void {
42      if ($e instanceof ApiError) {
43          $body = ['ok' => false, 'error' => $e->getMessage()];
44          if ($e->fields) {
45              $body['errors'] = $e->fields;
46          }
47          respond($e->getCode(), $body);
48      }
49      error_log('Practical 13 API error: ' . get_class($e) . ' [' . $e->getCode() . '] ' . $e-
     >getMessage());
50      respond(500, ['ok' => false, 'error' => 'The server could not complete the request. Please try
     again later.']);
51  });
52  set_error_handler(function (int $severity, string $message, string $file, int $line): bool {
```

`validate_event()` checks every field before any INSERT or UPDATE and returns all problems at once (HTTP 422 with an `errors` object, one message per field). The queries use MySQLi prepared statements; the search escapes the user's `%` and `_` so they are searched literally.

api.php (lines 188–215) – list with search and category filter (prepared statement)

```
188      case 'list':
189          require_method('GET');
190          $q = is_string($_GET['q'] ?? null) ? mb_substr(trim($_GET['q']), 0, 100) : '';
191          $category = is_string($_GET['category'] ?? null) && in_array($_GET['category'], CATEGORIES,
     true) ? $_GET['category'] : '';
192          $sql = 'SELECT ' . EVENT_COLUMNS . ' FROM events WHERE 1 = 1';
193          $types = '';
194          $params = [];
195          if ($q !== '') {
196              $like = '%' . addcslashes($q, '\\%_') . '%';             // the user's % and _ are
     searched literally
197              $sql .= ' AND (title LIKE ? OR venue LIKE ? OR category LIKE ? OR description LIKE ?)';
198              $types .= 'ssss';
199              array_push($params, $like, $like, $like, $like);
200          }
201          if ($category !== '') {
202              $sql .= ' AND category = ?';
203              $types .= 's';
204              $params[] = $category;
205          }
206          $stmt = $db->prepare($sql . ' ORDER BY event_date, start_time, event_id');
207          if ($types !== '') {
208              $stmt->bind_param($types, ...$params);
209          }
210          $stmt->execute();
211          $events = array_map('format_event', $stmt->get_result()->fetch_all(MYSQLI_ASSOC));
212          $total = $db->prepare('SELECT COUNT(*) FROM events');
213          $total->execute();
214          respond(200, ['ok' => true, 'count' => count($events), 'total' => (int) $total->get_result()-
     >fetch_row()[0],
215                        'query' => $q, 'category' => $category, 'categories' => CATEGORIES, 'events' =>
     $events]);
```

api.php (lines 229–238) – update: id check, 404, validation, prepared UPDATE

```
229      case 'update':
230          require_method('POST');
231          $body = json_body();
232          $id = event_id($body['event_id'] ?? null);
233          find_event($db, $id);                                        // 404 if it no longer exists
234          $e = validate_event($body);
235          $stmt = $db->prepare('UPDATE events SET title = ?, description = ?, category = ?, event_date =
     ?, start_time = ?, venue = ?, capacity = ?, status = ? WHERE event_id = ?');
236          $stmt->bind_param('ssssssisi', $e['title'], $e['description'], $e['category'],
     $e['event_date'], $e['start_time'], $e['venue'], $e['capacity'], $e['status'], $id);
237          $stmt->execute();
238          respond(200, ['ok' => true, 'message' => 'Event “' . $e['title'] . '” was updated.', 'event'
     => find_event($db, $id)]);
```

5.2 Fetch API front end (script.js)

All requests go through `apiRequest()`. It builds the URL, sends JSON for POST requests, parses the JSON answer and turns any non-OK answer into a JavaScript error that carries the server's message and the per-field errors.

script.js (lines 14–43) – the Fetch wrapper

```
14  async function apiRequest(action, { method = 'GET', params = {}, body = null, signal } = {}) {
15      const url = new URL(API, window.location.href);
16      url.searchParams.set('action', action);
17      Object.entries(params).forEach(([k, v]) => { if (v !== '' && v != null) url.searchParams.set(k,
     v); });
18      const options = { method, signal, headers: { Accept: 'application/json' } };
19      if (body !== null) {
20          options.headers['Content-Type'] = 'application/json';
21          options.headers['X-Requested-With'] = 'fetch';
22          options.body = JSON.stringify(body);
23      }
24      let response;
25      try {
26          response = await fetch(url, options);
27      } catch (err) {
28          if (err.name === 'AbortError') throw err;
29          throw new Error('The server could not be reached. Check that the PHP server is running.');
30      }
31      let data;
32      try {
33          data = await response.json();
34      } catch {
35          throw new Error(`The server sent an invalid response (HTTP ${response.status}).`);
36      }
37      if (!response.ok || !data.ok) {
38          const error = new Error(data.error || `Request failed (HTTP ${response.status}).`);
39          error.fields = data.errors || {};
40          error.status = response.status;
41          throw error;
42      }
43      return data;
```

After a successful create or update the form closes, a green message appears and the list is re-read with a GET request, the changed row highlighted; after a delete the row is removed. Data is always inserted with `textContent` (never `innerHTML`), so HTML in a title is shown as text. While a request runs, a loading indicator is shown and the table is dimmed; errors appear as a red message and next to the fields.

script.js (lines 190–215) – add/update without reloading the page

```
190  async function saveEvent(event) {
191      event.preventDefault();
192      clearFieldErrors();
193      const data = Object.fromEntries(new FormData($('event-form')).entries());
194      const isEdit = data.event_id !== '';
195      const errors = clientErrors(data);
196      if (Object.keys(errors).length) {
```

```
197          showFieldErrors(errors);
198          showMessage('Please correct the highlighted fields.', 'error');
199          return;
200      }
201      if (!isEdit) delete data.event_id;
202      const button = $('save-btn');
203      button.disabled = true;
204      setLoading(true, isEdit ? 'Saving changes…' : 'Adding event…');
205      try {
206          const result = await apiRequest(isEdit ? 'update' : 'create', { method: 'POST', body: data });
207          closeForm();
208          showMessage(result.message, 'success');
209          await loadEvents(result.event.event_id);
210      } catch (err) {
211          showFieldErrors(err.fields || {});
212          showMessage(err.message, 'error');
213      } finally {
214          button.disabled = false;
215          setLoading(false);
```

5.3 AJAX search with debounce

The search box sends no request per key press. Each key press restarts a 350 ms timer; only when the user pauses is the request sent. A still-running older search is cancelled with an `AbortController`, so results never arrive in the wrong order. In the browser test, typing “lab” key by key produced exactly one request (`q=lab`).

script.js (lines 134–137) – debounce

```
134  // Debounce: wait until the user stops typing for 350 ms before sending the search request
135  function onSearchInput() {
136      clearTimeout(searchTimer);
137      searchTimer = setTimeout(() => loadEvents(), 350);
```

6. Fetch API Request and Response Examples

The following exchanges were recorded from the running API (the same requests that `script.js` sends):

GET api.php?action=list&q=robotics

```
{"ok":true,"count":1,"total":12,"query":"robotics","category":"","categories":["Cultural","Hackathon","Semina
      r","Sports","Technical","Workshop"],"events":[{"event_id":6,"title":"Robotics
      Expo","description":"Line-following robots, drones and robotic arms built by student
      teams.","category":"Technical","event_date":"2027-01-09","start_time":"11:00","venue":"Exhibition
      Hall","capacity":400,"status":"open","created_at":"2026-07-06 10:00:00","updated_at":"2026-07-06
      10:00:00"}]}
[HTTP 200]
```

POST api.php?action=create

```
{"ok":true,"message":"Event “Hack Night 2027” was added.","event":{"event_id":13,"title":"Hack Night
      2027","description":"An overnight mini-hackathon with the Fetch
      API.","category":"Hackathon","event_date":"2027-03-06","start_time":"19:00","venue":"Innovation Lab,
      Block A","capacity":80,"status":"open","created_at":"2026-10-30 20:14:38","updated_at":"2026-10-30
      20:14:38"}}
[HTTP 200]
```

POST api.php?action=update

```
{"ok":true,"message":"Event “Hack Night 2027” was updated.","event":{"event_id":13,"title":"Hack Night
      2027","description":"An overnight mini-hackathon with the Fetch
      API.","category":"Hackathon","event_date":"2027-03-06","start_time":"20:00","venue":"Innovation Lab,
      Block A","capacity":90,"status":"open","created_at":"2026-10-30 20:14:38","updated_at":"2026-10-30
      20:14:38"}}
[HTTP 200]
```

POST api.php?action=delete

```
{"ok":true,"message":"Event “Hack Night 2027” was deleted.","deleted_id":13}
[HTTP 200]
```

POST api.php?action=create (invalid)

```
{"ok":false,"error":"Please correct the highlighted fields.","errors":{"title":"Title must be 3–120
      characters.","event_date":"Enter a valid date (YYYY-MM-DD).","capacity":"Capacity must be a whole
      number from 1 to 5000."}}
[HTTP 422]
```

The browser sends the POST requests with `Content-Type: application/json`, `X-Requested-With: fetch` and the event as a JSON string in the body (`JSON.stringify`).

7. Validation and Security

- **Input validation** on the server for every field before any database operation (422 with field messages); a lighter check in the browser for quick feedback.
- **SQL injection:** MySQLi prepared statements with `bind_param()` for every query; no `query()` call in the code.
- **Safe output:** JSON encoded with `JSON_HEX_TAG|AMP|APOS|QUOT`; the page uses `textContent`; Content-Security-Policy in `index.html`.
- **Correct methods:** reading only with GET, changing only with POST (405 with an `Allow` header otherwise); POST needs a JSON body and the `X-Requested-With` header, which a cross-site HTML form cannot send.
- **Central JSON error handling:** one exception handler; database and PHP errors are logged, the client receives a general message.
- **Private files:** `schema.sql`, `README.md`, config files, tests and screenshots answer HTTP 403; credentials belong in the git-ignored `config.local.php`.

8. Output Screenshots

All screenshots are genuine captures of the running application in Microsoft Edge with the fictional demo data. Figures 1–9 were taken during the browser test (headless Edge); the loading state was produced with real network throttling (2.5 s latency via the DevTools protocol). Figures 10–12 show the real Edge DevTools connected to the app (with a live screencast of the page on the left): the Network panel lists the genuine `fetch` requests from `script.js` to `api.php`, and each request's real JSON response was opened with DevTools' *Open in Sources panel*.

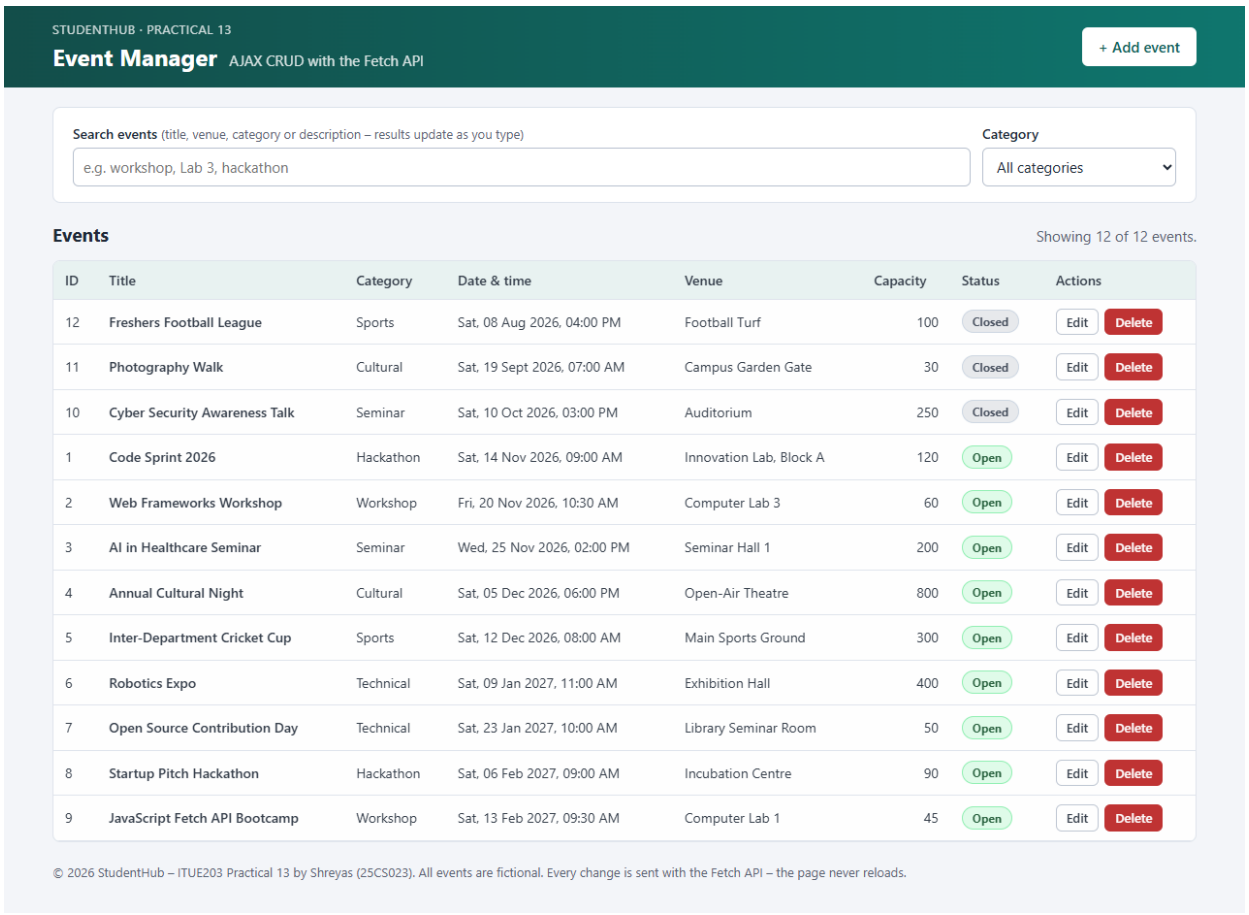


Figure 1: Event list loaded with fetch(GET api.php?action=list) – 12 events (event-list.png)

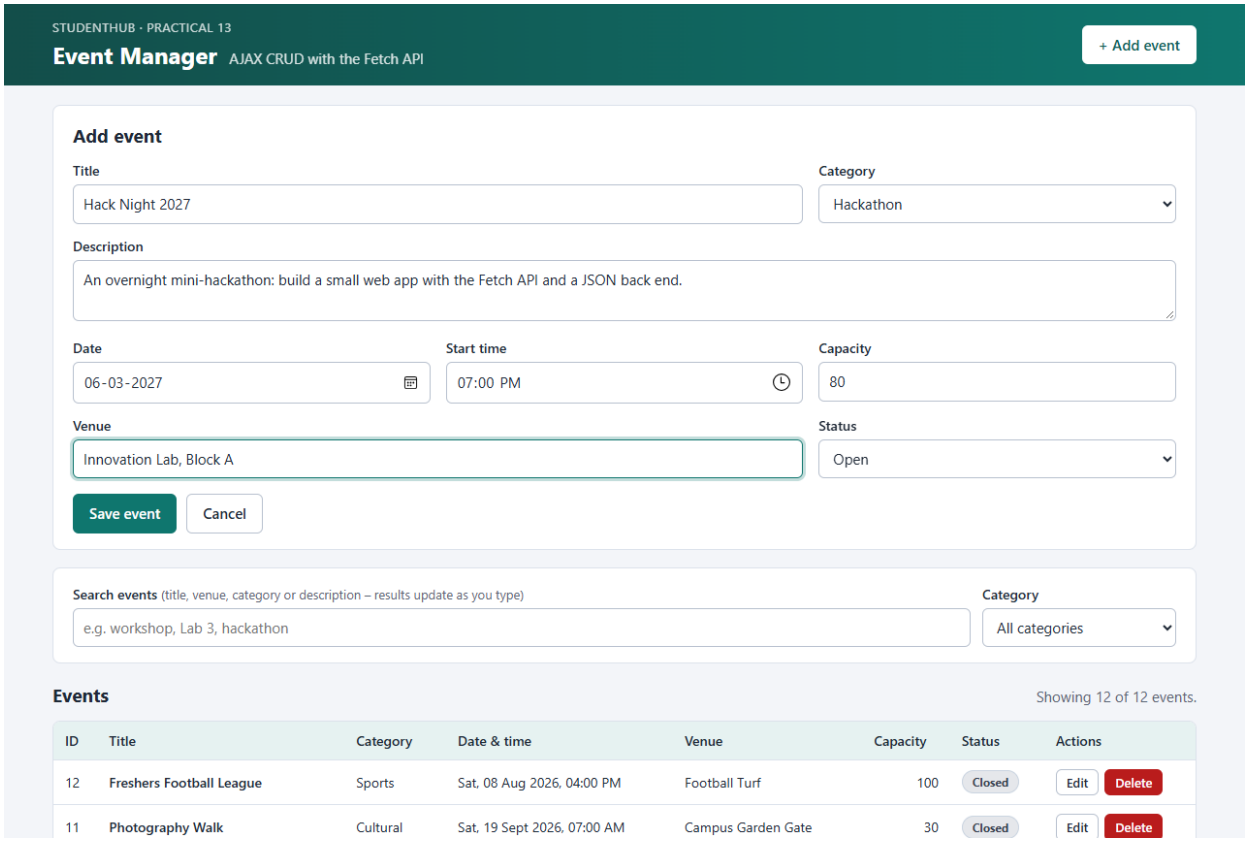


Figure 2: Add event form filled in (Hack Night 2027) (add-event-form.png)

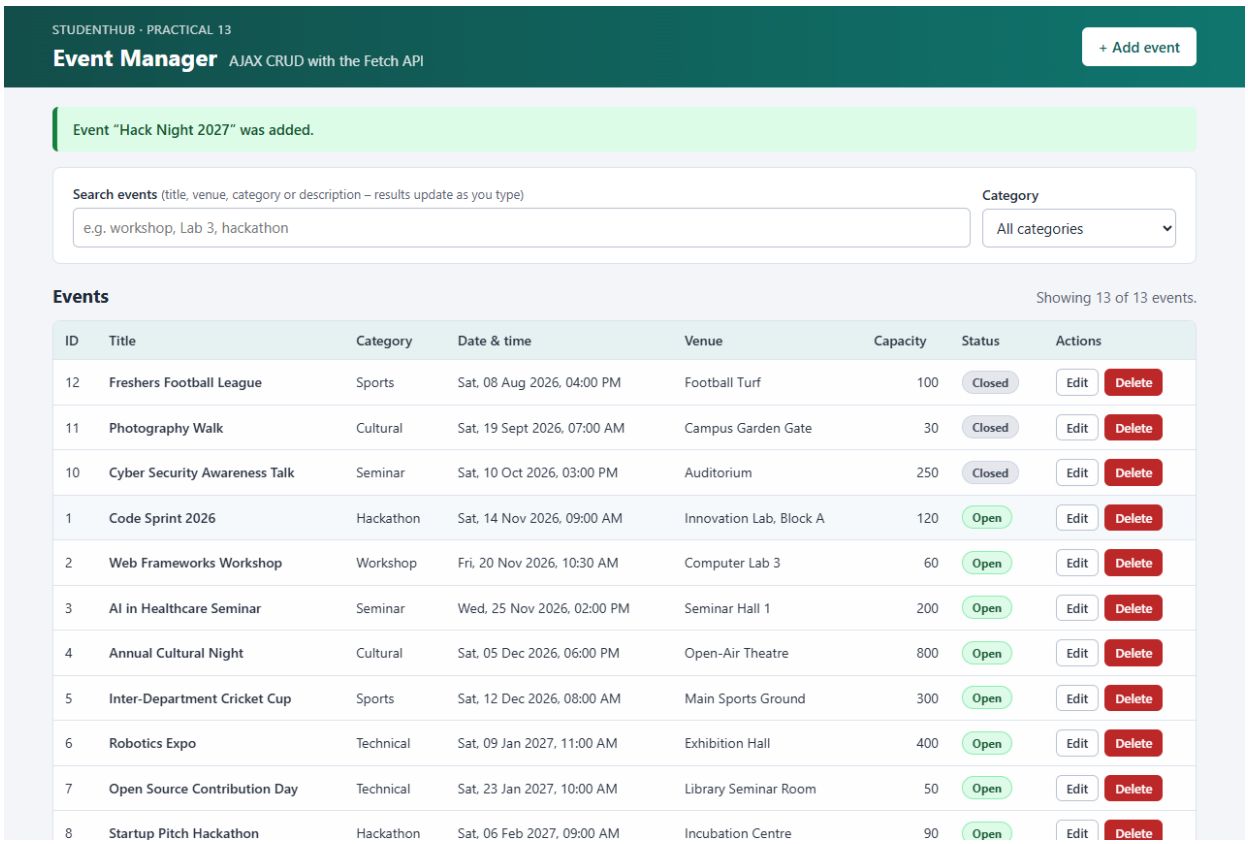


Figure 3: Event added without reloading – success message and new row highlighted (successful-add.png)

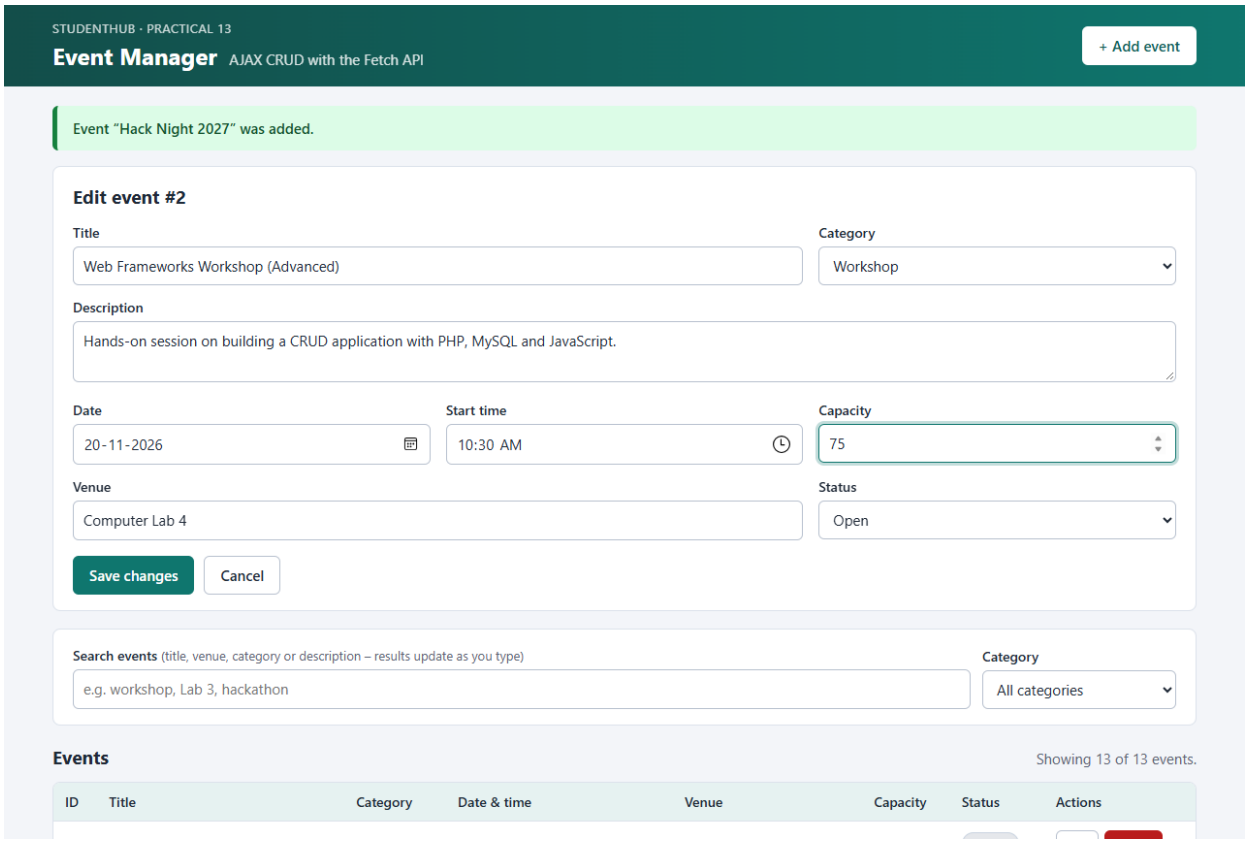


Figure 4: Edit form for "Web Frameworks Workshop" with new title, venue and capacity (edit-event-form.png)

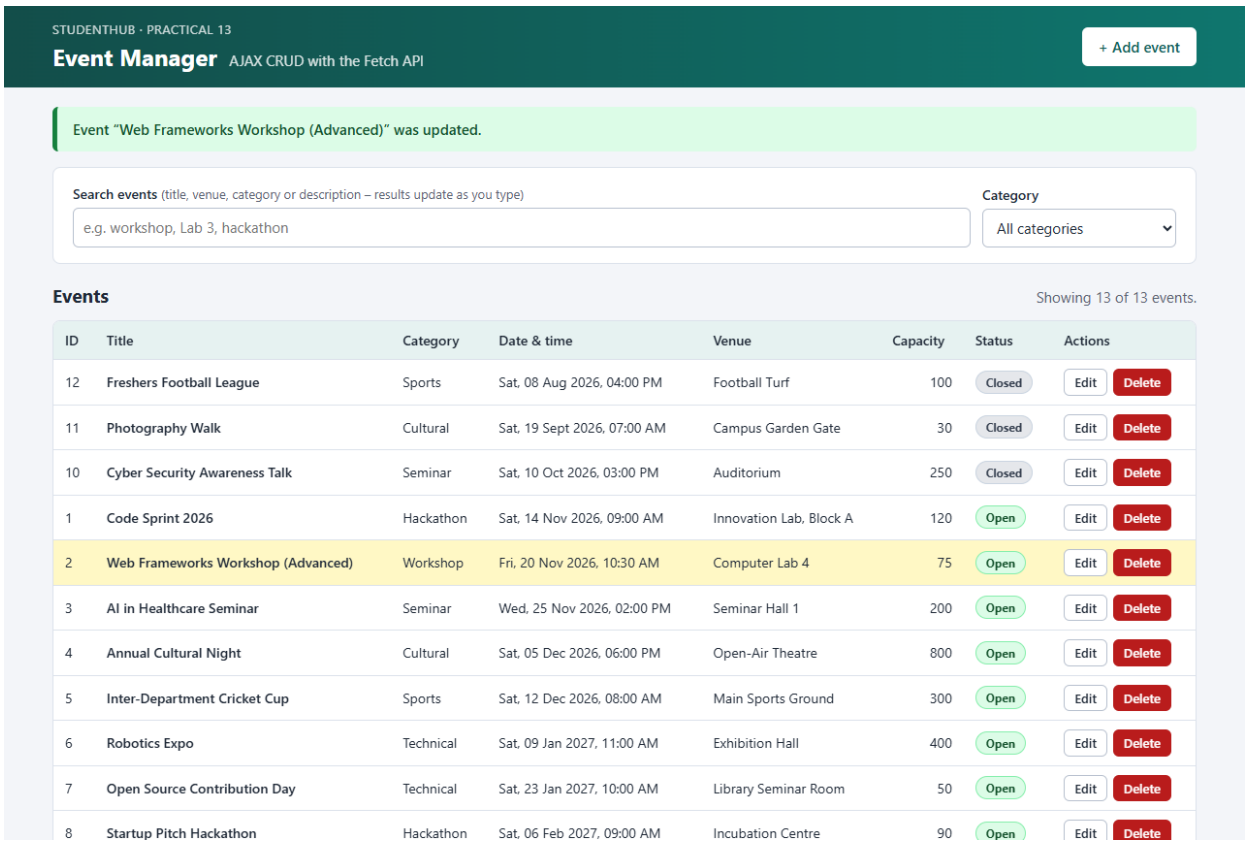


Figure 5: Event updated without reloading – message and changed row highlighted (successful-update.png)

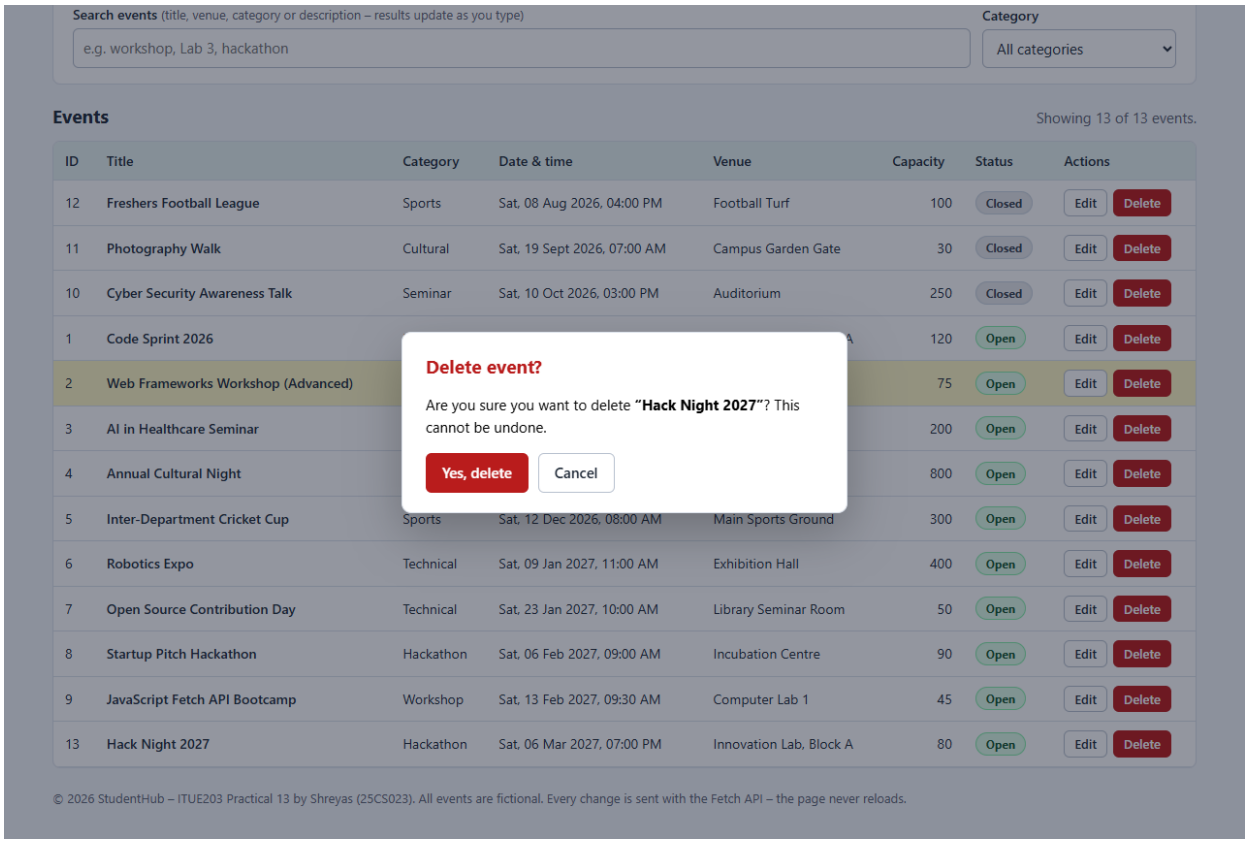


Figure 6: Confirmation dialog before deleting "Hack Night 2027" (delete-confirmation.png)

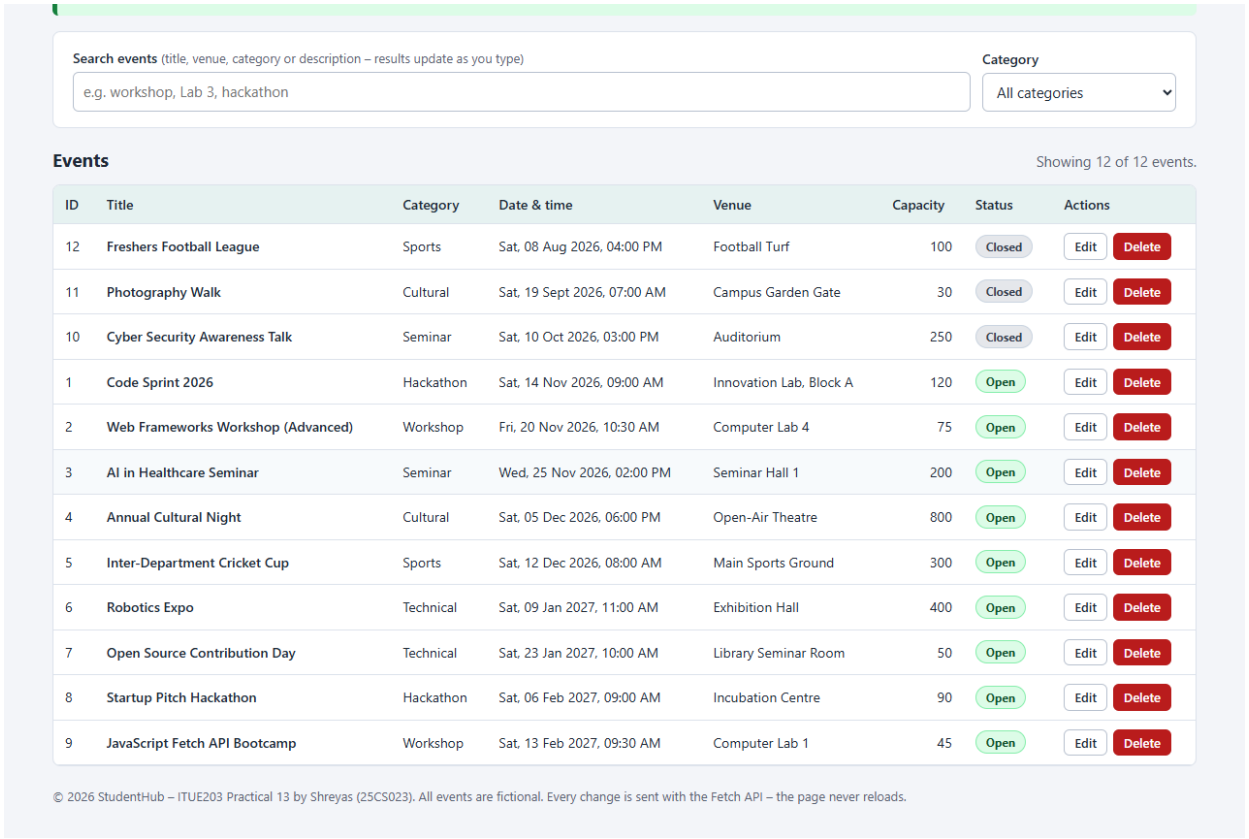


Figure 7: Event deleted without reloading – row removed, 12 events again (successful-delete.png)

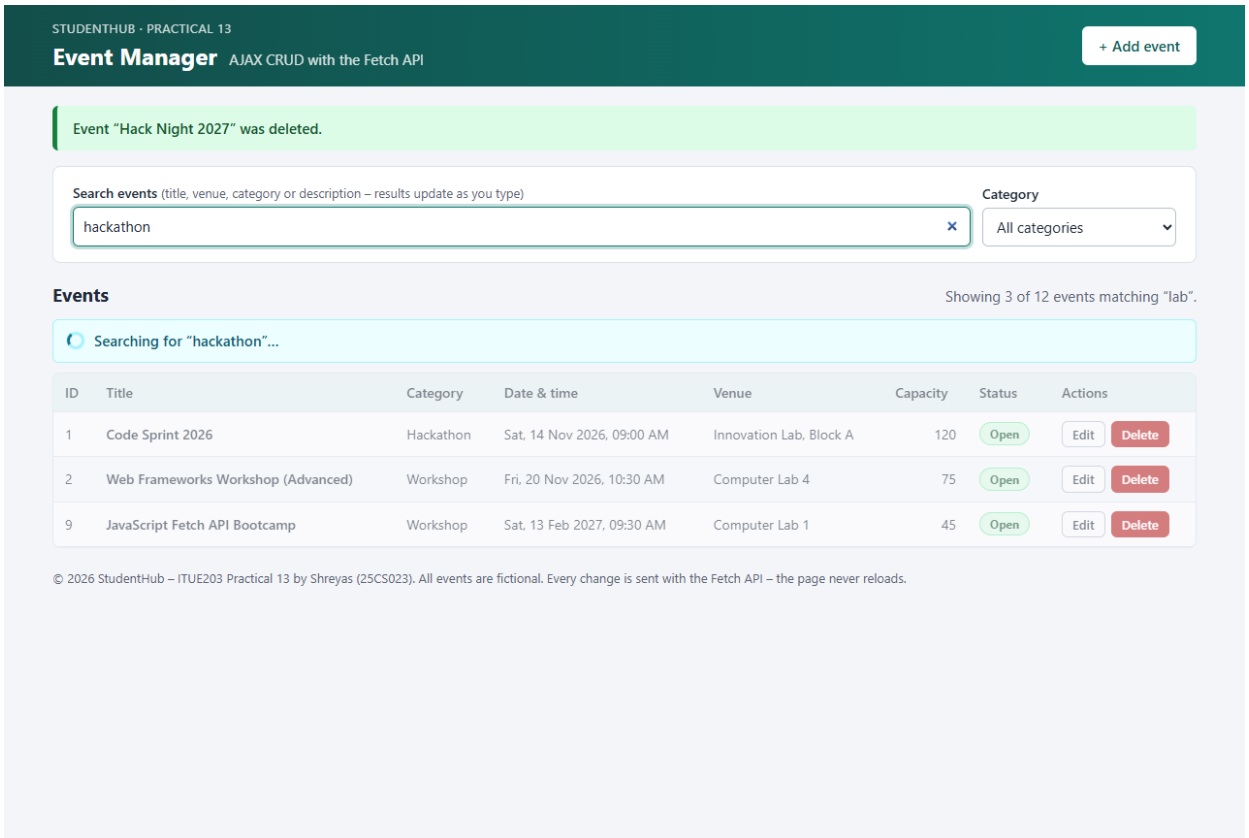


Figure 8: Loading state – "Searching for 'hackathon'…" while the throttled request runs (loading-state.png)

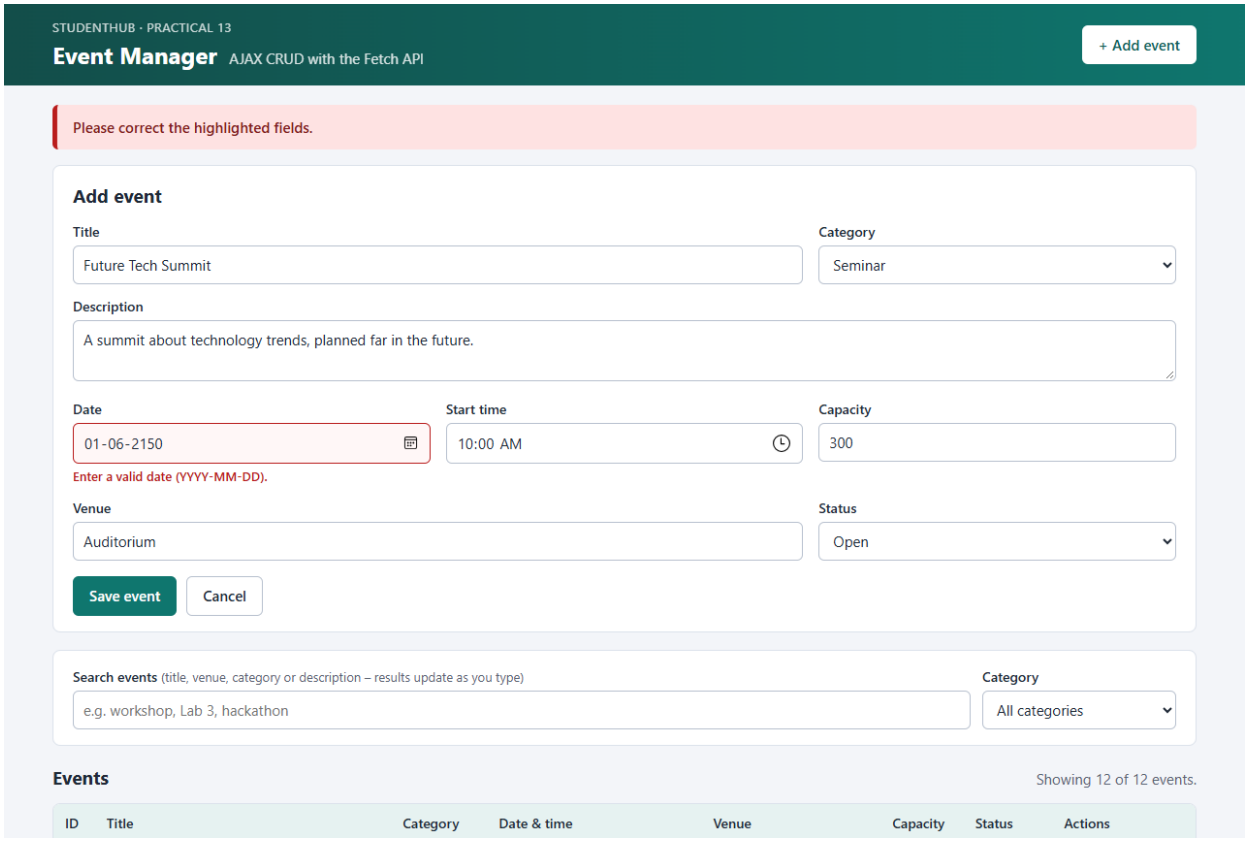


Figure 9: Error state – the API rejects a date after 2100 (HTTP 422) with a field message (error-state.png)

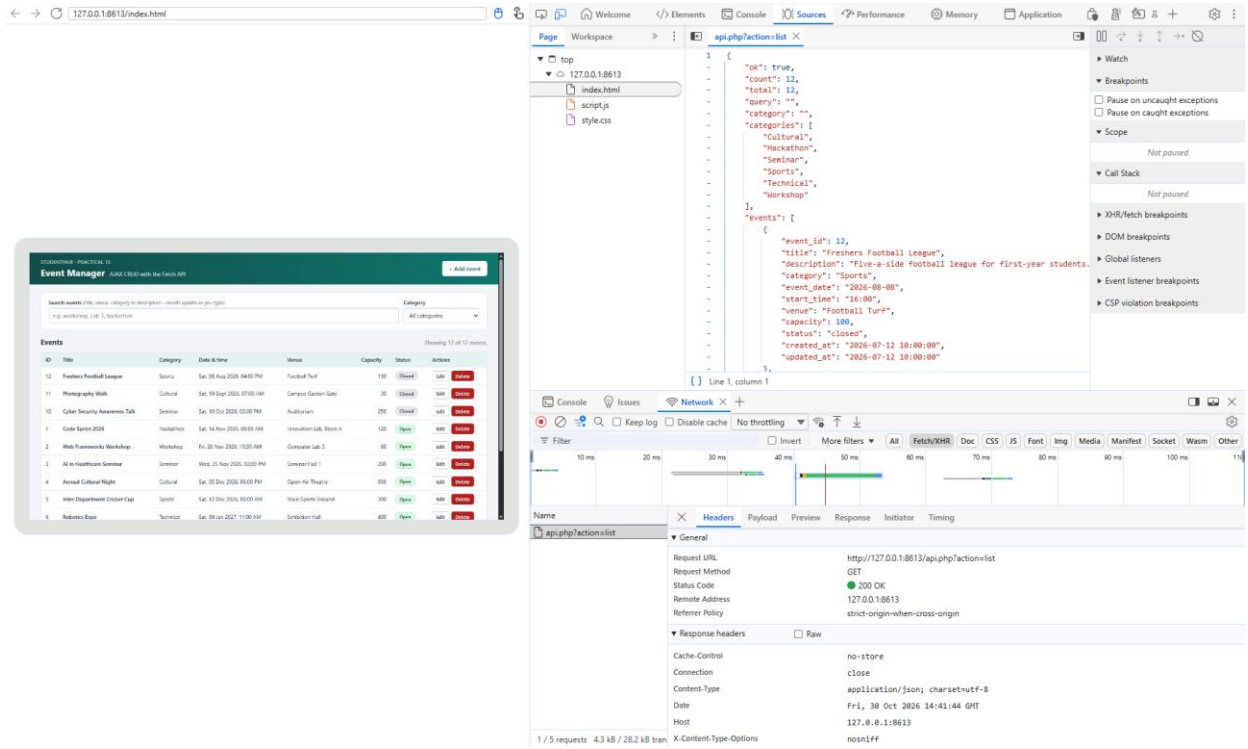


Figure 10: Network: GET api.php?action=list – 200 OK, application/json, JSON response (network-get-json.png)

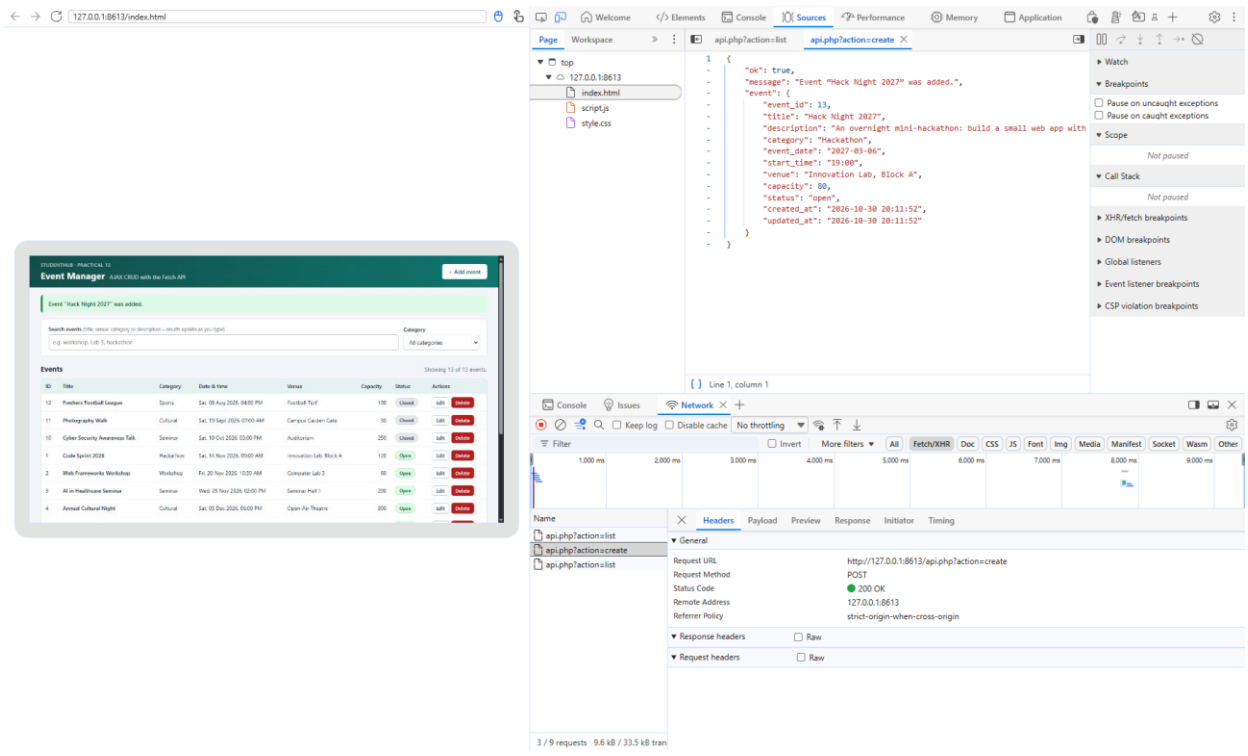


Figure 11: Network: POST api.php?action=create – 200 OK and the JSON response with the new event (network-post-add.png)

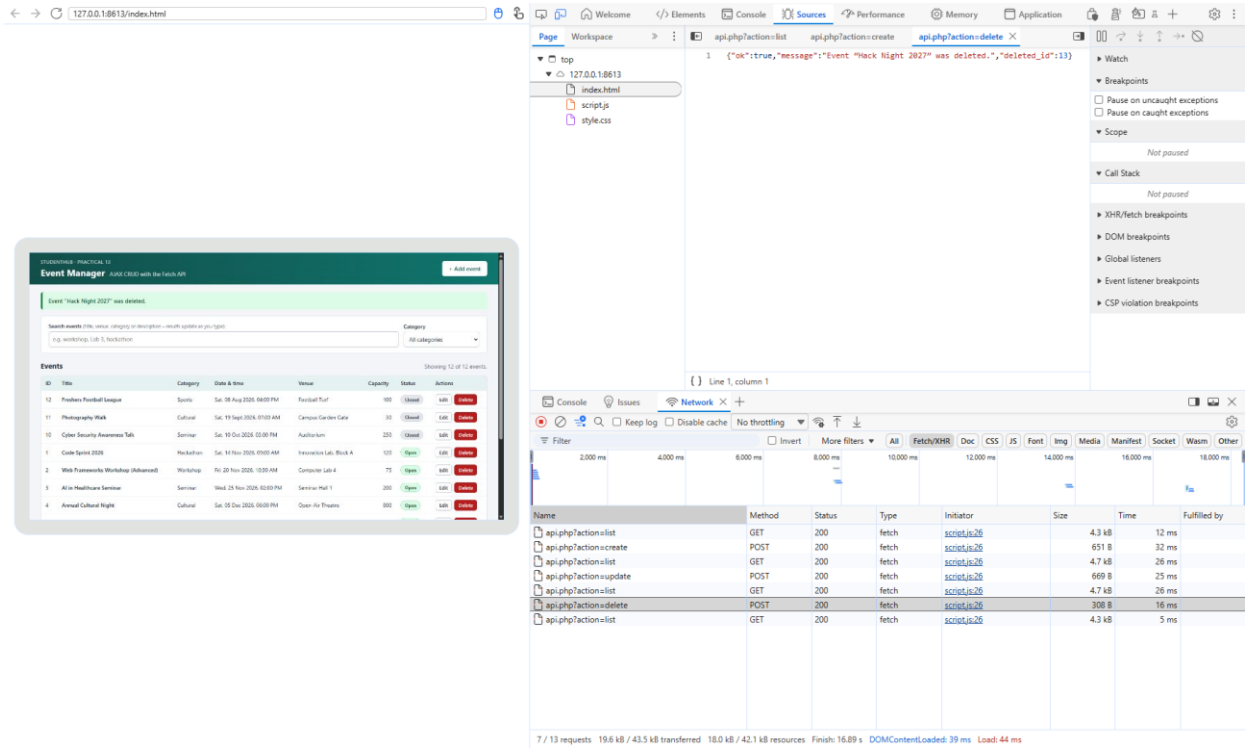


Figure 12: Network: list, create, update and delete fetch requests (Method, Status 200) and the delete JSON response (network-crud-operations.png)

9. Test Cases and Results

`tests/test_app.py` (Python standard library) imports `schema.sql` into the disposable database `studenthub_prac13_test`, starts two PHP servers (normal; MySQL unreachable) and checks the source files and every API action, including validation, wrong methods and bodies, SQL injection, XSS and the central

error handling. `tests/browser_screenshots.py` checks the page in Microsoft Edge: Fetch requests, no page reload, confirmation, debounce, loading and error states.

Result: 89 of 89 automated API checks passed, 0 failed; 13 of 13 browser checks passed. `php -l` found no syntax errors in the 4 PHP files.

ID	Test case	Expected result	Result
TC01	Required files	all five exist in `Practical-13/`	Pass
TC02	Database import	12 events (9 open, 3 closed), InnoDB, indexes, CHECK constraints enforced	Pass
TC03	List events (API)	200, `application/json`, `{"ok":true,"count":12,…}`, numeric ids, sorted by date	Pass
TC04	List events (page)	table filled by a `fetch` GET request, 12 rows	Pass
TC05	Search by title / venue / category	matching events only, case-insensitive	Pass
TC06	Search edge cases	0 results; wildcards searched literally	Pass
TC07	Category filter	only Sports; combined filter; unknown value ignored	Pass
TC08	Debounced AJAX search	only **one** request (`q=lab`) after typing stops; results updated	Pass
TC09	Get one event	200 with event; 404; 400 JSON errors	Pass
TC10	Add event (API)	200 `{"ok":true,"message":"…was added.","event":{…}}`, row stored, title spaces cleaned	Pass
TC11	Add event without reload (page)	POST `create`, success message, new row highlighted, page not reloaded	Pass
TC12	Required fields	422 with errors for all 8 fields	Pass
TC13	Invalid values	422 with the field error; nothing stored	Pass
TC14	Update event (API)	200, row changed, saved event returned	Pass
TC15	Update without reload (page)	POST `update`, row shows new values, page not reloaded	Pass
TC16	Update errors	422 (row unchanged); 404; 400	Pass
TC17	Delete confirmation	dialog “Are you sure you want to delete …?”	Pass
TC18	Delete without reload	POST `delete`, 200 `{"ok":true,…,"deleted_id":…}`, row removed, page not reloaded	Pass
TC19	Delete errors	404 “not found”; 400	Pass
TC20	Loading state	“Searching for …” indicator visible, hidden after the response	Pass
TC21	Error state	HTTP 422 shown as message and field error “Enter a valid date”	Pass
TC22	Wrong method	405 JSON with `Allow: POST` / `Allow: GET`	Pass
TC23	Wrong body	415; 400; 400; 400; 400; 400 – all JSON	Pass
TC24	SQL injection	0 results; stored literally, table intact	Pass
TC25	XSS	stored as text; JSON escapes < as <; page shows it as text, no element created, no alert	Pass
TC26	Central JSON error handling	500 `{"ok":false,"error":"The server could not complete…"}`, no SQL details; real error in PHP log	Pass
TC27	Protected files	403	Pass
TC28	Network evidence	`fetch` requests to `api.php`, 200 OK, JSON responses (screenshots 10–12)	Pass

10. Problems Encountered and Fixes

- **JSON body type:** the first test run showed that a JSON array (`[1,2,3]`) was accepted as a request body. The API now checks that the body is a JSON object before using it.
- **Test timing:** in the browser test the “added” message was still visible when the update finished, so the first wait returned too early; the test now waits for the “was updated” text.
- **Network screenshots:** capturing a visible Edge window would have used the real desktop while it was in use, so the DevTools front end served by Edge was opened in a headless browser instead – still the

genuine Edge DevTools, recording genuine requests. Because a selected request hides the Status column, the response was opened in the Sources panel while the Network panel shows the status.

11. Conclusion

The event manager lists, adds, updates and deletes events entirely with the Fetch API: the page never reloads, `api.php` answers every request in JSON with a correct status code, GET and POST are separated, and all input is validated before a prepared statement touches the database. Loading, success and error states keep the user informed, and the debounced search sends one request per pause instead of one per key press. The Edge Network panel confirms the fetch requests and their JSON responses; 89 automated API checks and 13 browser checks passed.

12. References

- MDN – Using the Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- MDN – AbortController: https://developer.mozilla.org/en-US/docs/Web/API/AbortController
- PHP Manual – json_encode: https://www.php.net/manual/en/function.json-encode.php
- PHP Manual – MySQLi prepared statements: https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php
- Microsoft Learn – Inspect network activity (Edge DevTools): https://learn.microsoft.com/microsoft-edge/devtools-guide-chromium/network/
- OWASP – REST Security Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html
- Project repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023 (folder Practical-13)